느리게 갱신되는 세그먼트 트리 (Segment Tree with Lazy Propagation)

문제

크기가 N인 정수 배열 A가 있고, 여기서 다음과 같은 연산을 최대 M번 수행해야 하는 문제가 있습니다.

1. 구간 l, r ($l \le r$)이 주어졌을 때, $A[l] + A[l+1] + \cdots + A[r-1] + A[r]$을 구해서 출력하기
2. i번째 수부터 j번째 수에 v를 더하기

세그먼트 트리에 설명한 문제에서 2번 연산은 i번째 수에 v를 더하는 것이었습니다. 이번 문제의 2번 연산은 구간에 수를 더하는 것입니다. 세그먼트 트리에서 수를 변경하는 연산을 i번째 수부터 j번째 수까지 하나씩 하는 방식을 이용할 수 있습니다. 1번 연산은 구간의 합을 구하는 연산이라 $O(\lg N)$입니다. 2번 연산은 수 하나를 변경하는 연산을 $j - i + 1$번 해야하니 $O(N \lg N)$입니다. 세그먼트 트리를 사용하지 않고 배열에서 1번, 2번 연산을 수행하는 경우 시간 복잡도는 $O(N)$인데, 오히려 시간이 더 걸립니다.

느리게 갱신되는 세그먼트 트리 설명을 위해 세그먼트 트리에서 구간을 변경하는 함수 `update_range`를 만들어봅시다.

소스 1

C++ Java Python

```python
def update_range(tree, node, start, end, left, right, diff):
    if left > end or right < start:
        return
    if start == end:
        tree[node] += diff
        return
    update_range(tree, node*2, start, (start+end)//2, left, right, diff)
    update_range(tree, node*2+1, (start+end)//2+1, end, left, right, diff)
    tree[node] = tree[node*2] + tree[node*2+1]

```

`update_range(tree, node, start, end, left, right, diff)`는 `left`번째부터 `right`번째 수에 `diff`를 더하는 소스입니다.

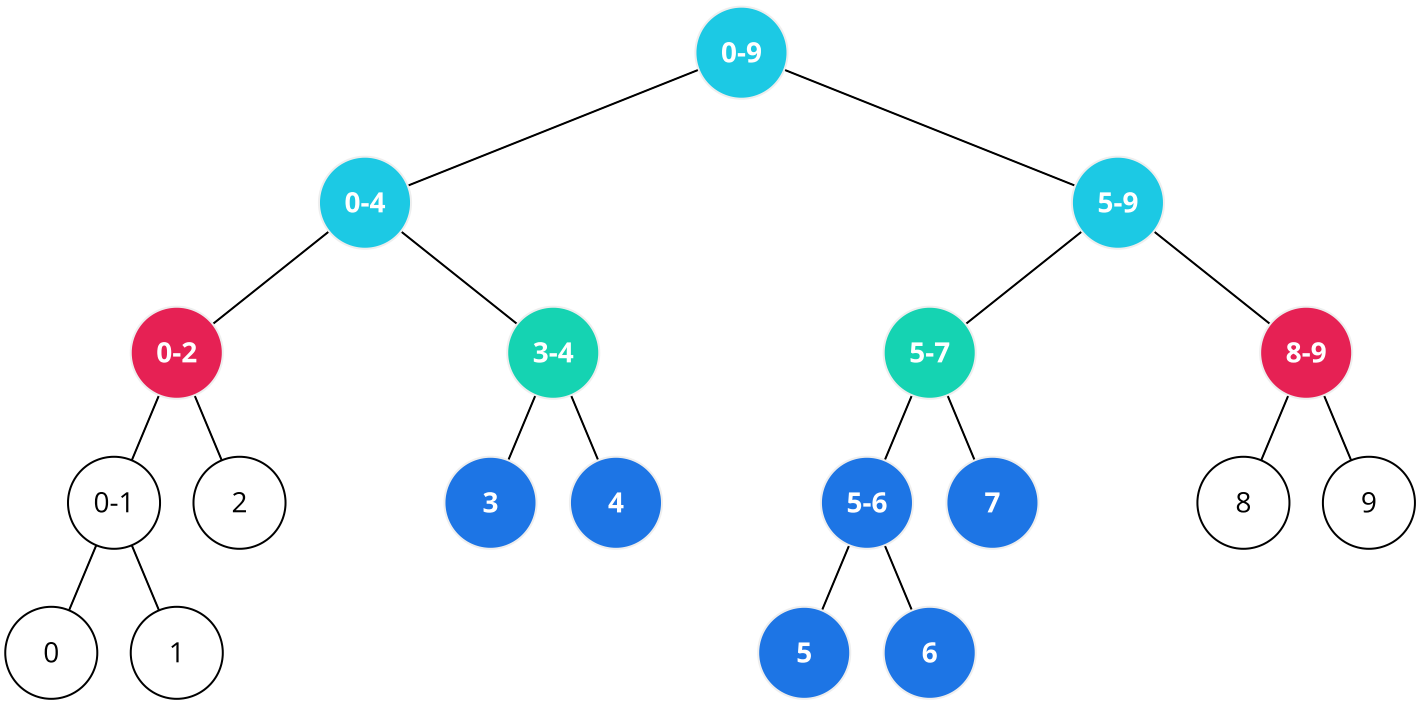
리프 노드를 찾아 `diff`를 더하고, 리프 노드가 아닌 나머지 노드가가 저장하는 합도 변경해 줍니다.

효율적으로 보이지만, 모든 수를 변경해야 하면 모든 노드를 다 변경해야 합니다. 따라서, 시간 복잡도는 $O(N \lg N)$입니다.

느리게 갱신되는 세그먼트 트리를 사용하면 구간 변경을 효율적으로 수행할 수 있습니다.

변경해야 하는 노드

다음은 $N =$ 10, `left =` 3, `right =` 7 인 경우 변경해야 하는 노드를 표시했습니다.



, , 은 `[left,right]`를 변경하기 위해서 합을 변경해야 하는 노드입니다.

여기서 을 루트로하는 서브트리는 모든 노드에 들어있는 합을 변경해야 합니다. 이런 경우 은 합을 변경하지 않고 나중에 다시 변경을 수행하러 그 노드에 방문했을 때 변경을 진행해도 됩니다.

lazy

나중에 변경해야 하는 값을 `lazy[i]`에 저장합시다.

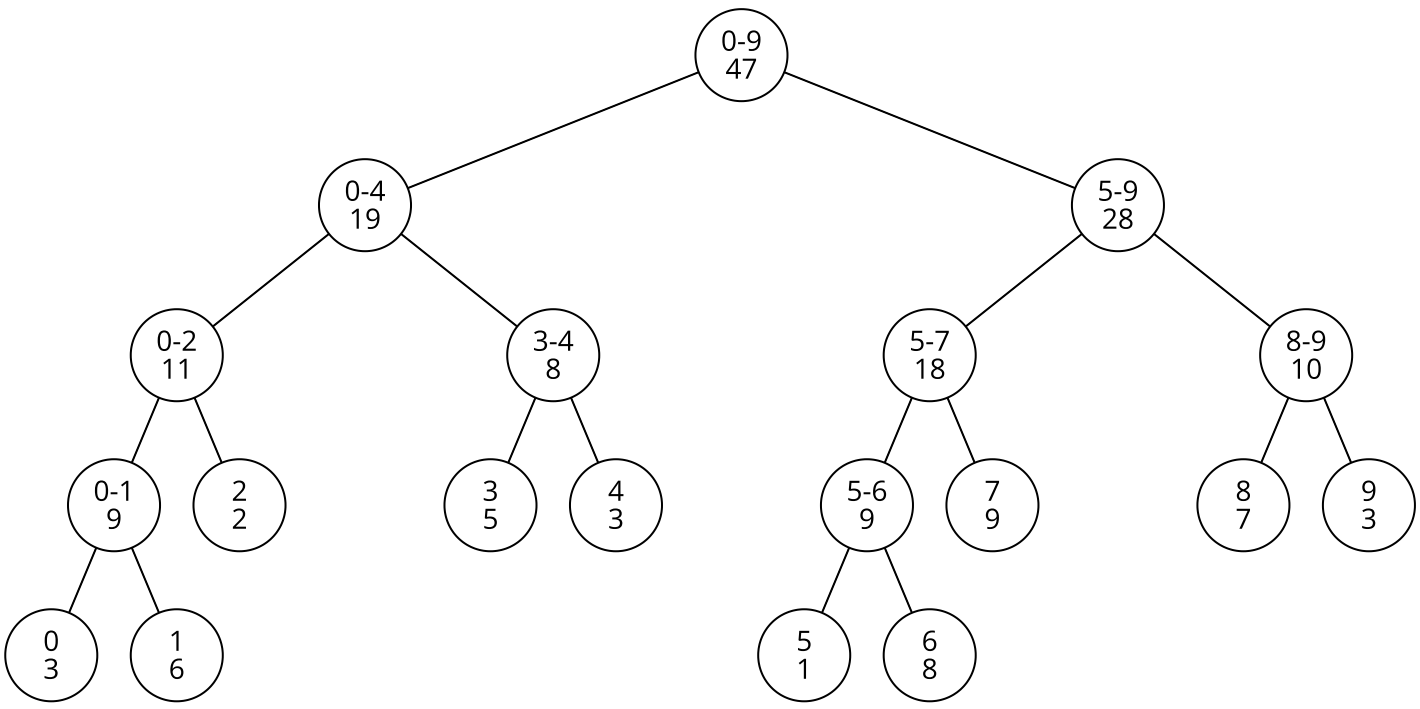
`[3,4]`의 합을 저장하는 노드의 `lazy[i]`에 10이 저장되어 있다면, $A[3]$과 $A[4]$에 10을 더해야 하는데, 나중에 10을 더하겠다는 의미를 갖습니다.

예시

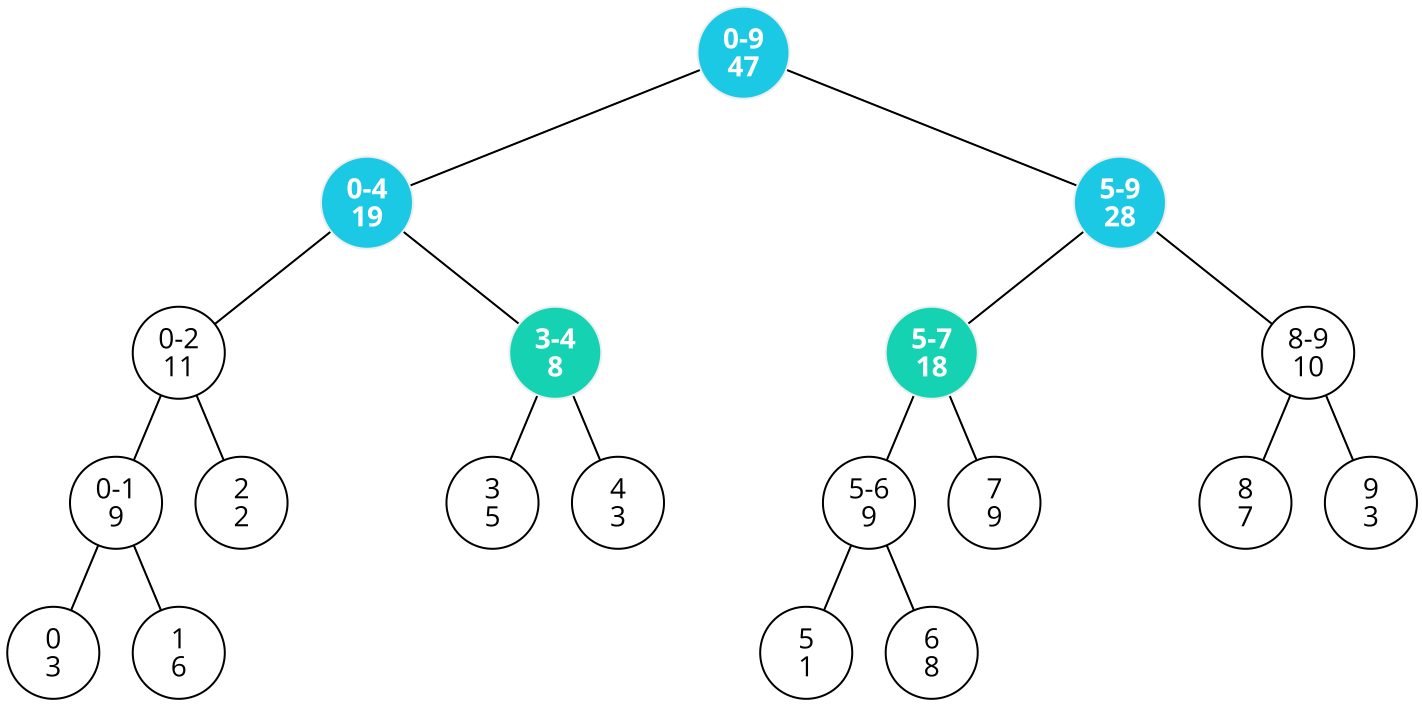
A가 다음과 같은 경우를 살펴봅시다.

i	0	1	2	3	4	5	6	7	8	9
$A[i]$	3	6	2	5	3	1	8	9	7	3

이 정보를 이용해서 세그먼트 트리를 만들어봅시다. 노드의 위에는 합을 저장하고 있는 구간, 아래에는 저장된 합이 있습니다.



여기서 `[3,7]`에 2를 더하는 경우 변경해야 하는 노드는 아래 그림의 와 입니다.



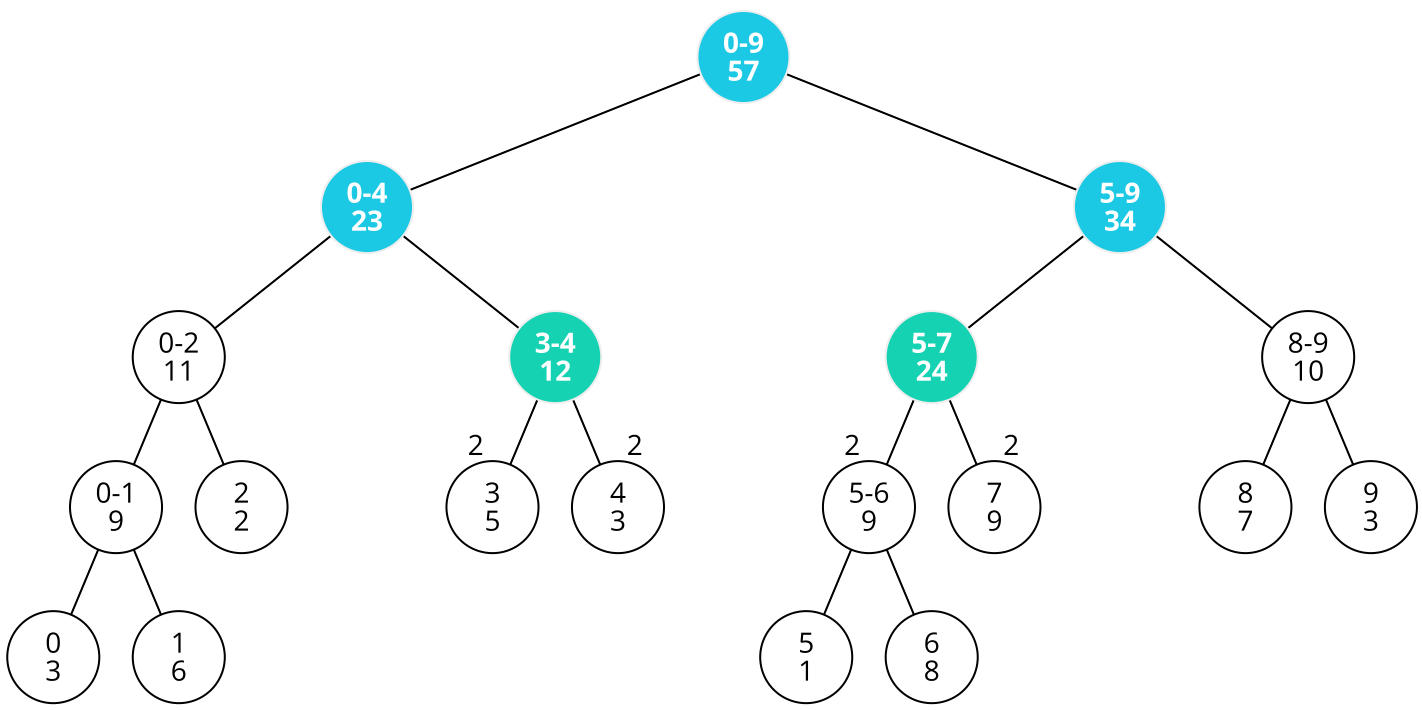
은 노드의 구간이 `[3,7]`에 일부만 포함되는 경우이고, 는 모두 포함되는 경우입니다.

은 세그먼트 트리에서 변경을 처리하는 과정과 똑같이 처리하면 되지만, 은 조금 특별하게 진행해야 합니다.

의 아래에 있는 노드는 모두 변경하려는 구간 `[3,7]`에 포함됩니다. 따라서, 이러한 노드의 변경은 나중에 필요할때 하기로 하고, 그 값을 `lazy[i]`에 저장합니다.

이제 A는 다음과 같고, 세그먼트 트리는 아래 그림과 같습니다. 세그먼트 트리 노드의 왼쪽 대각선 상단 또는 오른쪽 대각선 상단에 있는 값은 그 노드의 `lazy[i]` 값입니다. `lazy[i]`가 0인 경우 표시하지 않습니다.

i	0	1	2	3	4	5	6	7	8	9
$A[i]$	3	6	2	7	5	3	10	11	7	3

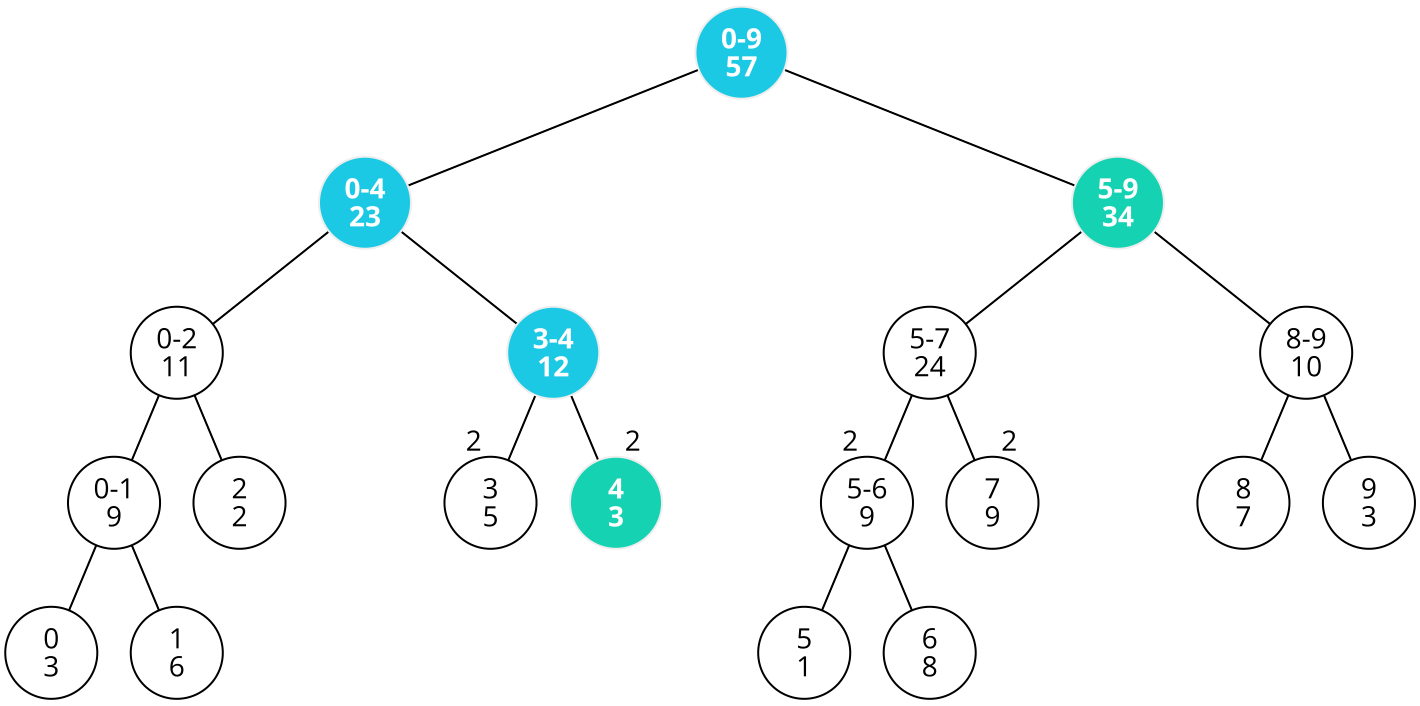


앞으로 어떤 노드를 방문할 때마다 `lazy[i]`값이 있는지 확인해야 합니다. `lazy[i]`값이 0이 아닌 경우에는 노드의 합을 변경하고, 자식 노드에게 `lazy[i]`값을 전달해야 합니다.

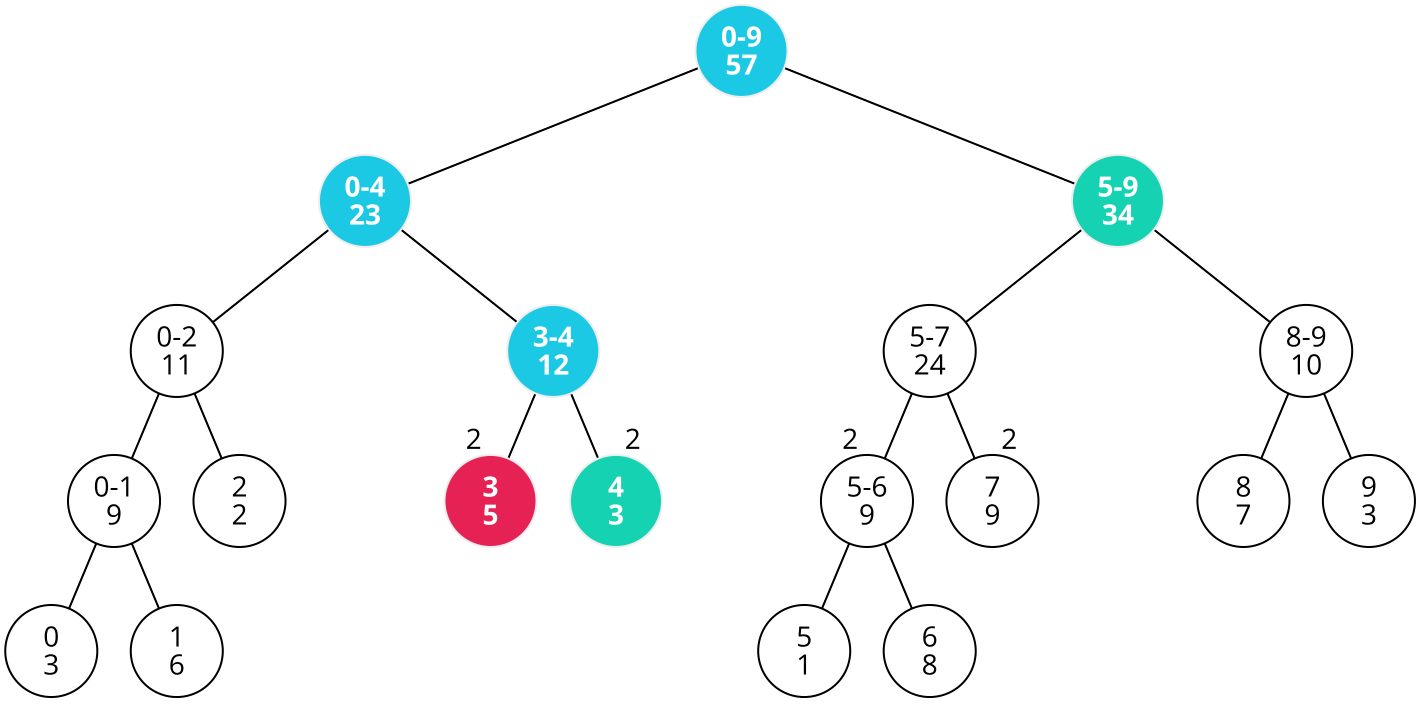
`lazy[i]`에는 그 노드가 담당하는 구간의 각 수에 더해야 하는 값이 저장되어 있으니, 합에는 그 구간에 포함된 수의 개수만큼 곱해서 더해야 합니다.

예를 들어, 어떤 노드 `node`에 구간 `[start,end]`의 합이 저장되어 있다면, `tree[node]`에는 `lazy[node]*(end-start+1)`을 더해야 합니다.

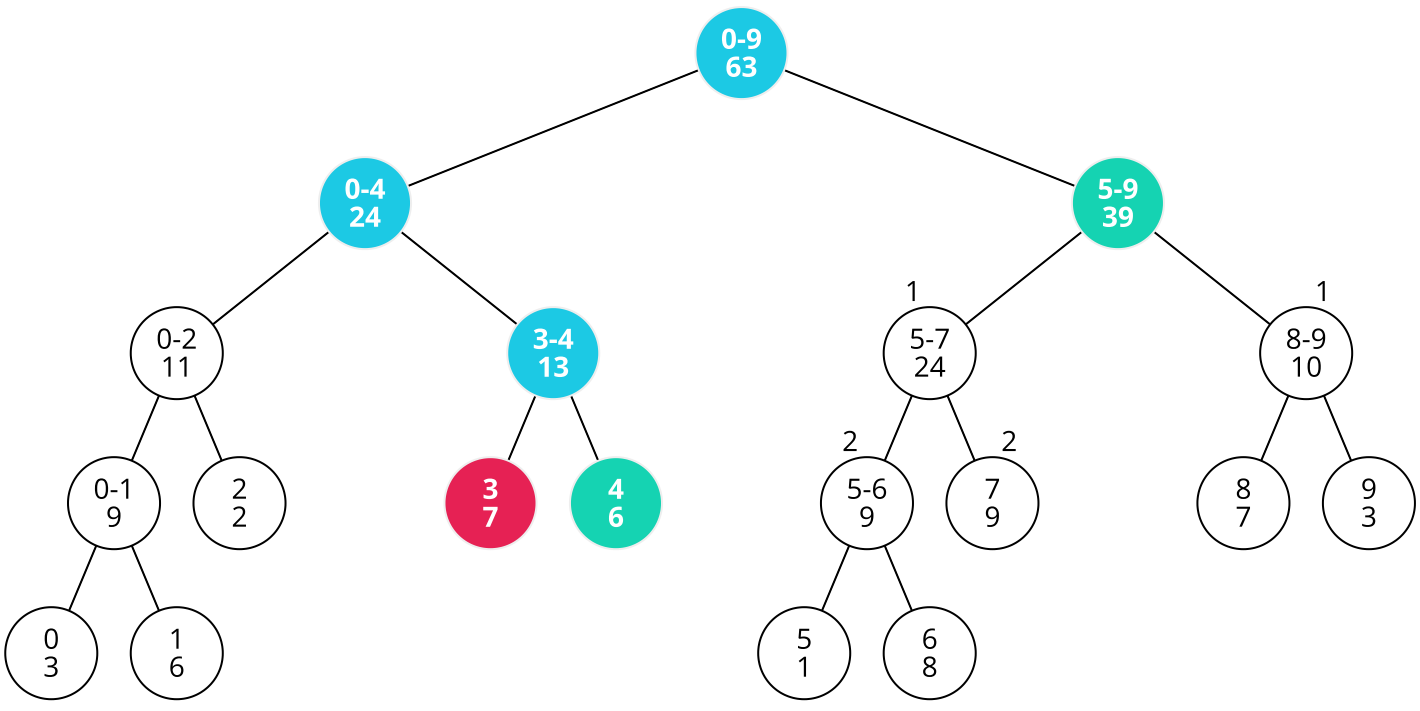
이제 `[4,9]`에 1을 더해봅시다. 여기서 변경해야 하는 노드를 표시해보면 다음과 같습니다.



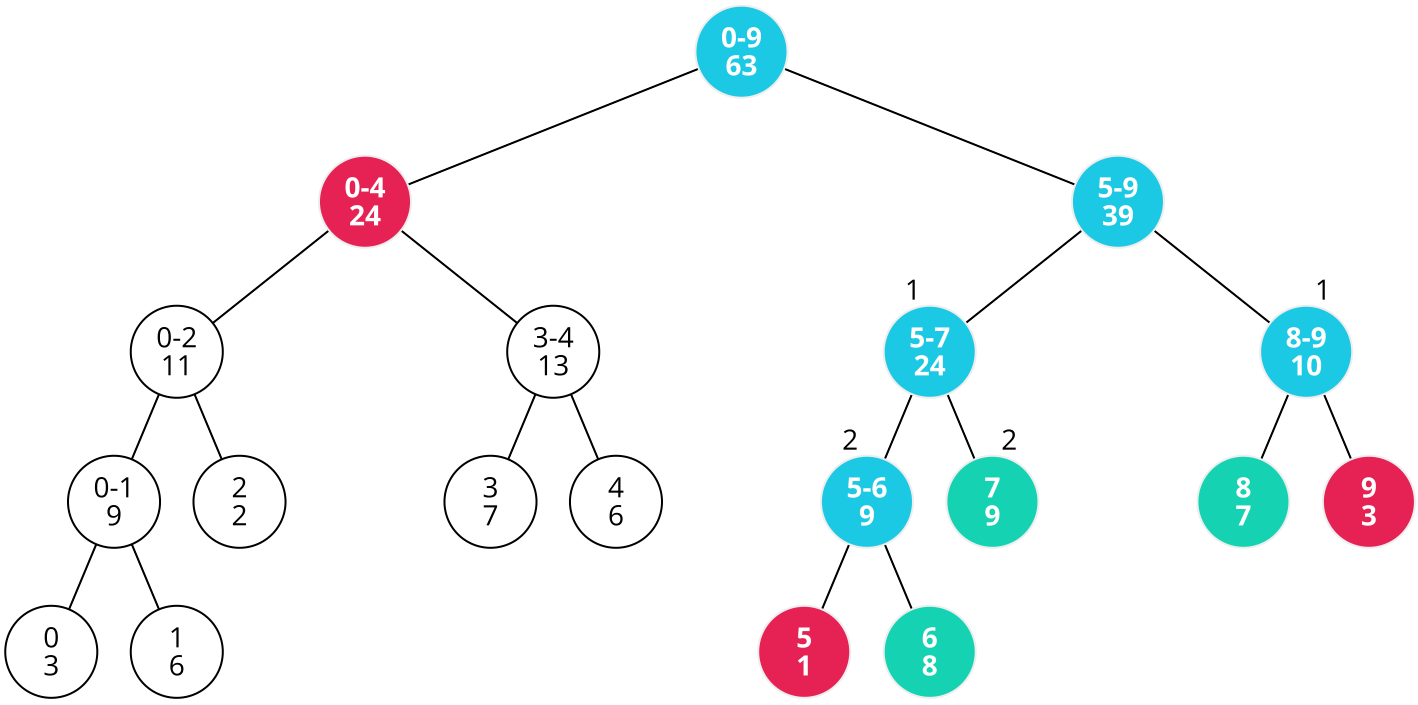
[3,4]의 합을 저장하고 있는 노드를 방문했을 때, 항상 두 자식 모두를 호출합니다. 따라서, [3,3]의 합을 저장하고 있는 노드도 호출하게 됩니다. 이 노드는 로 표시했습니다.



[3,3]의 합과 [4,4]의 합을 저장하고 있는 노드를 방문했을 때는 lazy값이 0보다 크기 때문에, lazy값을 이용해 합을 변경하고, 그 다음에 1을 더해 주게 됩니다. 트리는 다음과 같이 변합니다.



마지막으로 [6,8]의 합을 구해보겠습니다. 합을 구하려면 아래 그림의 색칠된 노드를 모두 방문하게 됩니다.



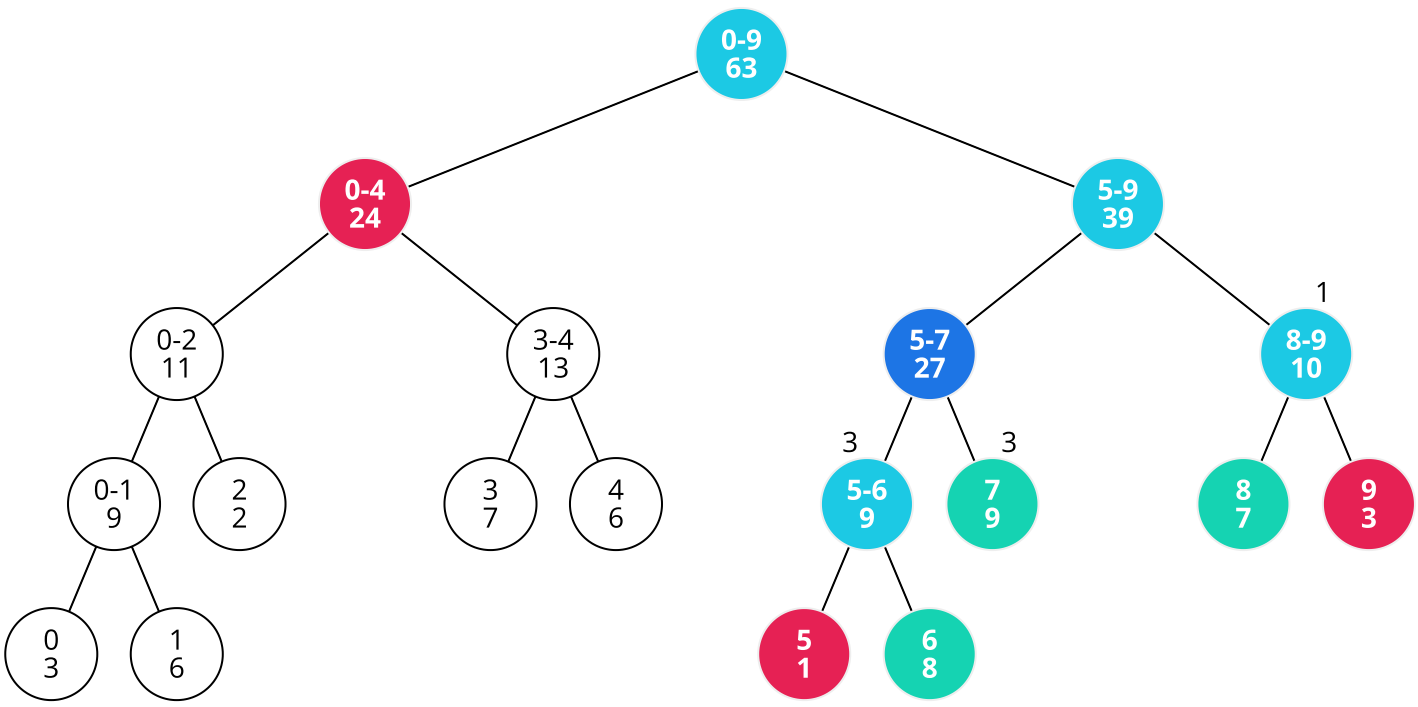
1. 가장 처음 루트 노드 0-9 63 입니다. `[0,9]`는 `[6,8]`와 겹치기 때문에, 자식 노드 0-4 24 와 5-9 39 를 호출합니다.

2. 0-4 24 의 구간 `[0,4]`는 `[6,8]`과 겹치지 않기 때문에, 0을 리턴합니다.

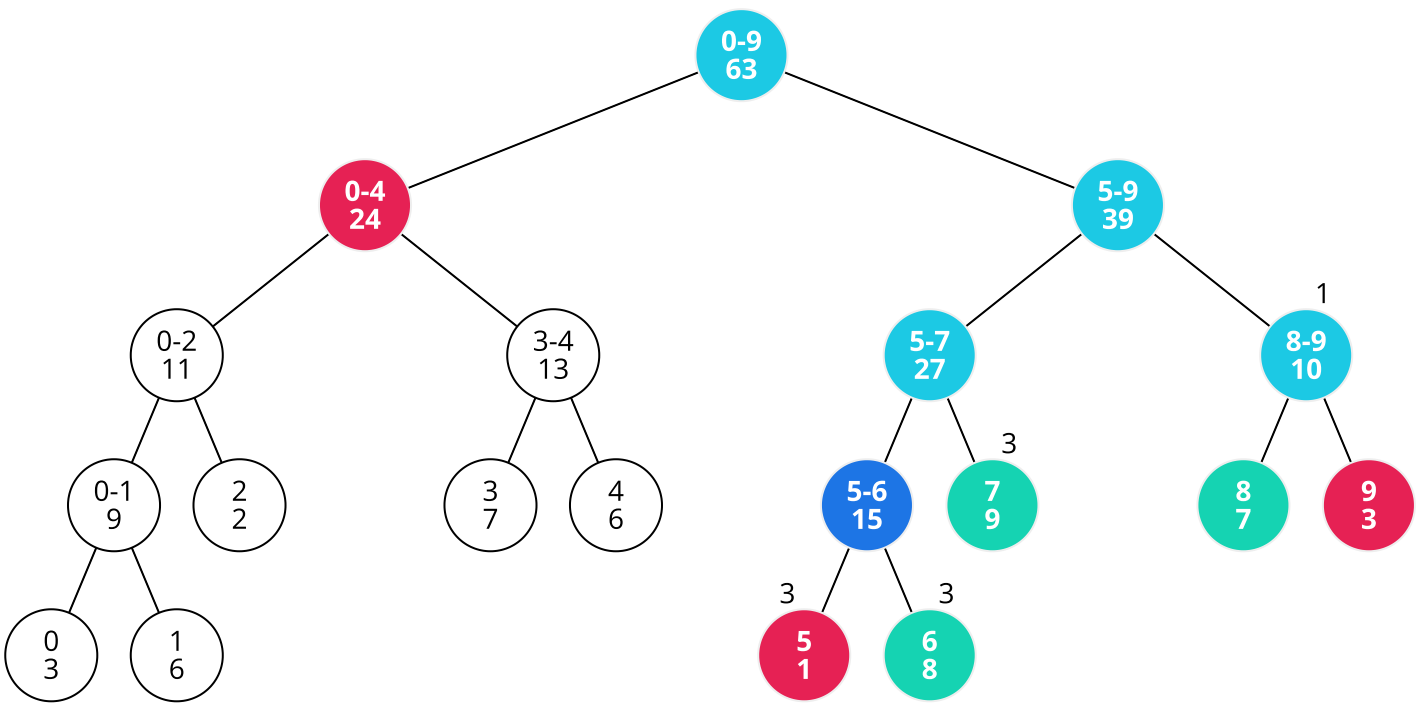
3. 5-9 39 의 구간 `[5,9]`는 `[6,8]`과 겹치기 때문에, 자식 노드 5-7 24 와 8-9 10 를 호출합니다.

4. 5-7 24 에는 `lazy`값이 있습니다. 먼저 현재 노드의 합을 변경해주고, 자식 노드 5-6 9 와 7 9 에 `lazy`값을 넘겨줍니다.

5-7 24 이 저장하고 있는 합은 24이고, `lazy`의 값은 1입니다. 구간의 크기는 $7-5+1=3$이기 때문에, 변경되는 합은 $24+(7-5+1)\times 1=27$이 됩니다. 자식 노드에게 `lazy`를 넘겨준 이후 트리는 다음과 같습니다.

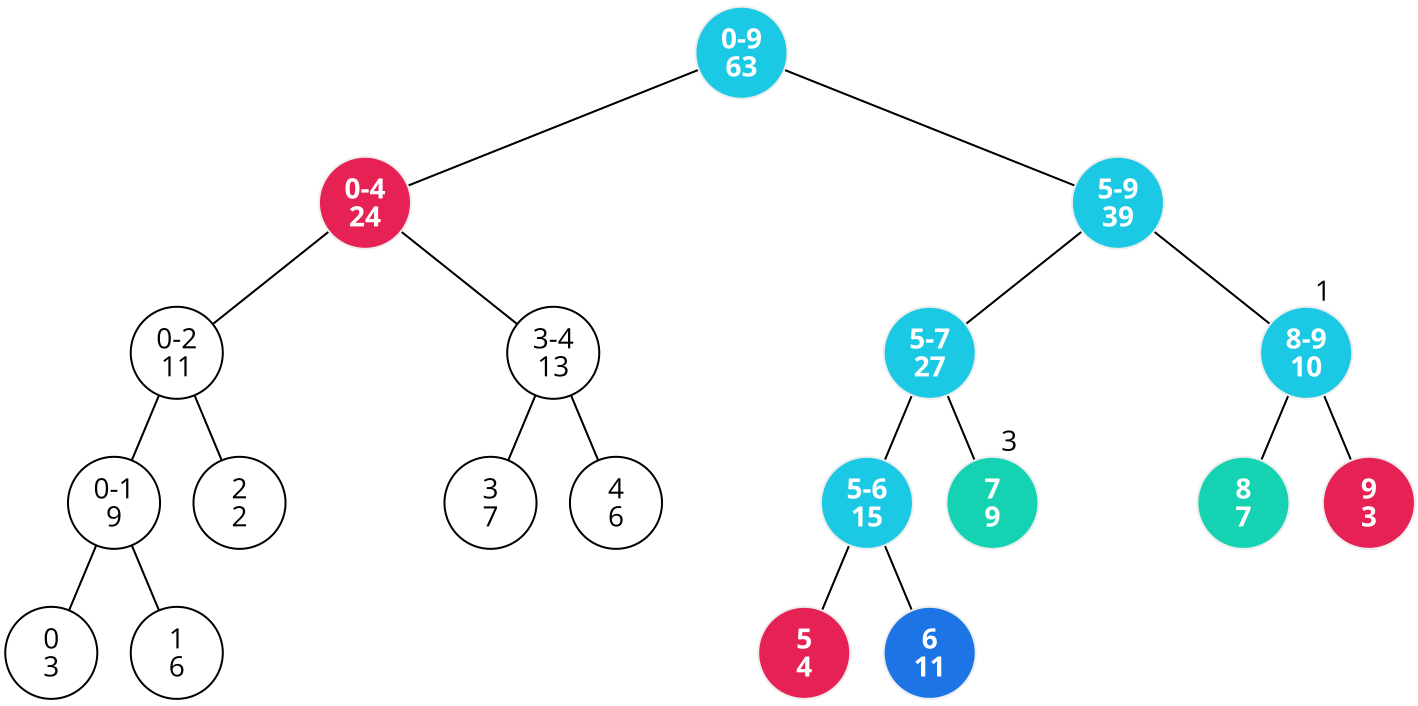


5. 5-6 9 에는 `lazy`값이 있습니다. 현재 노드의 합을 변경해 5-6 15 가 되고, 두 자식 노드에게 `lazy`를 넘겨줍니다.

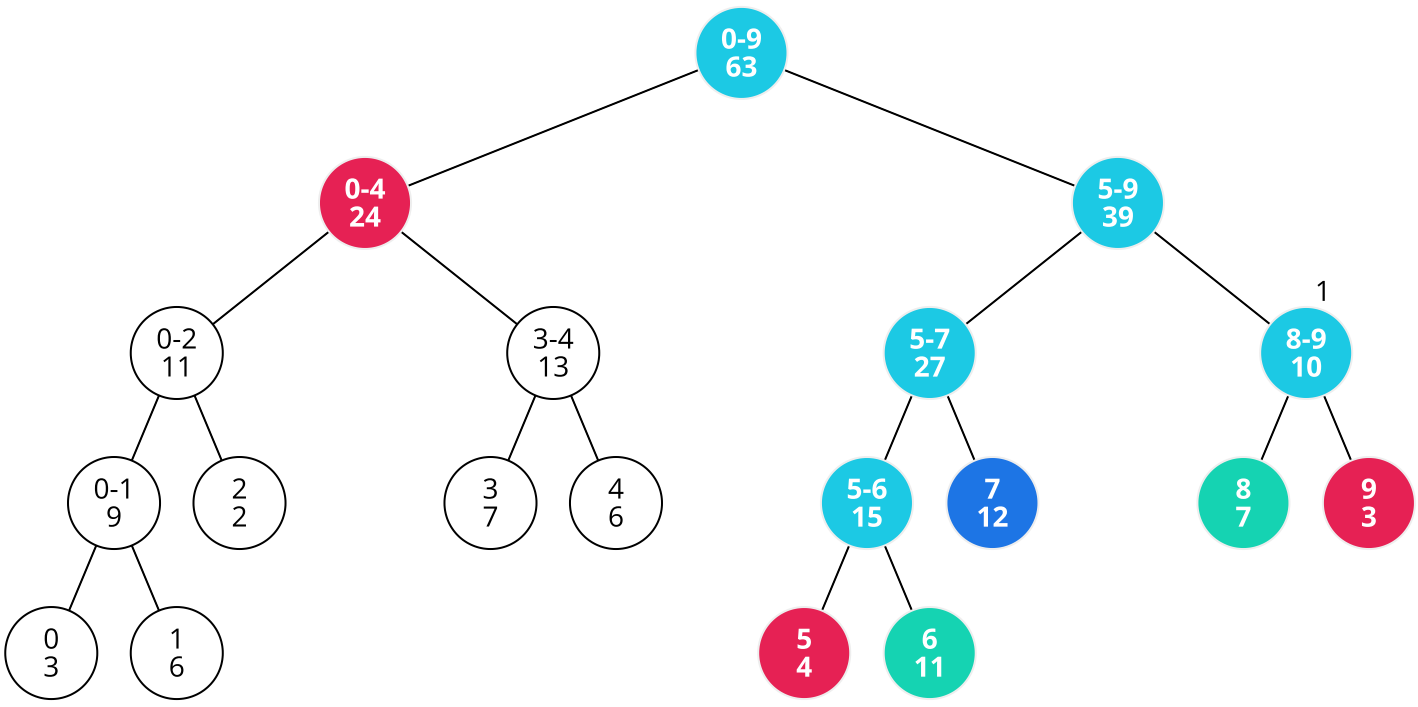


6. 5 1 에는 `lazy`값이 있고, 먼저 합을 변경합니다. 합을 구하려는 구간 `[6,8]`와 겹치지 않으니 0을 리턴합니다.

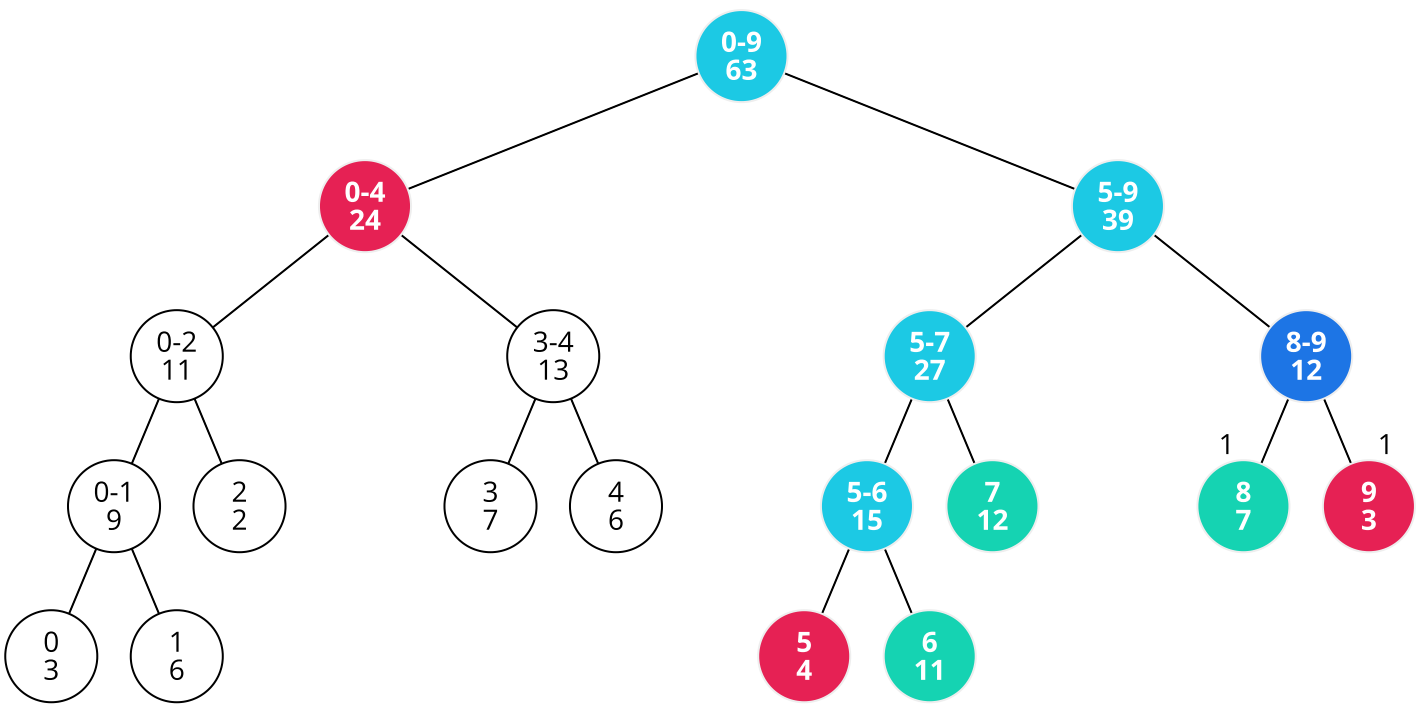
7. 6 8 에도 `lazy`값이 있으니 합을 변경해 6 11 가 됩니다. `[6,8]`에 포함되니 11을 리턴합니다.



8. 7 9 는 `lazy`값도 있고, 구간 `[6,8]`에도 포함됩니다. 합을 변경해 7 12 가 되고, 12를 리턴합니다.



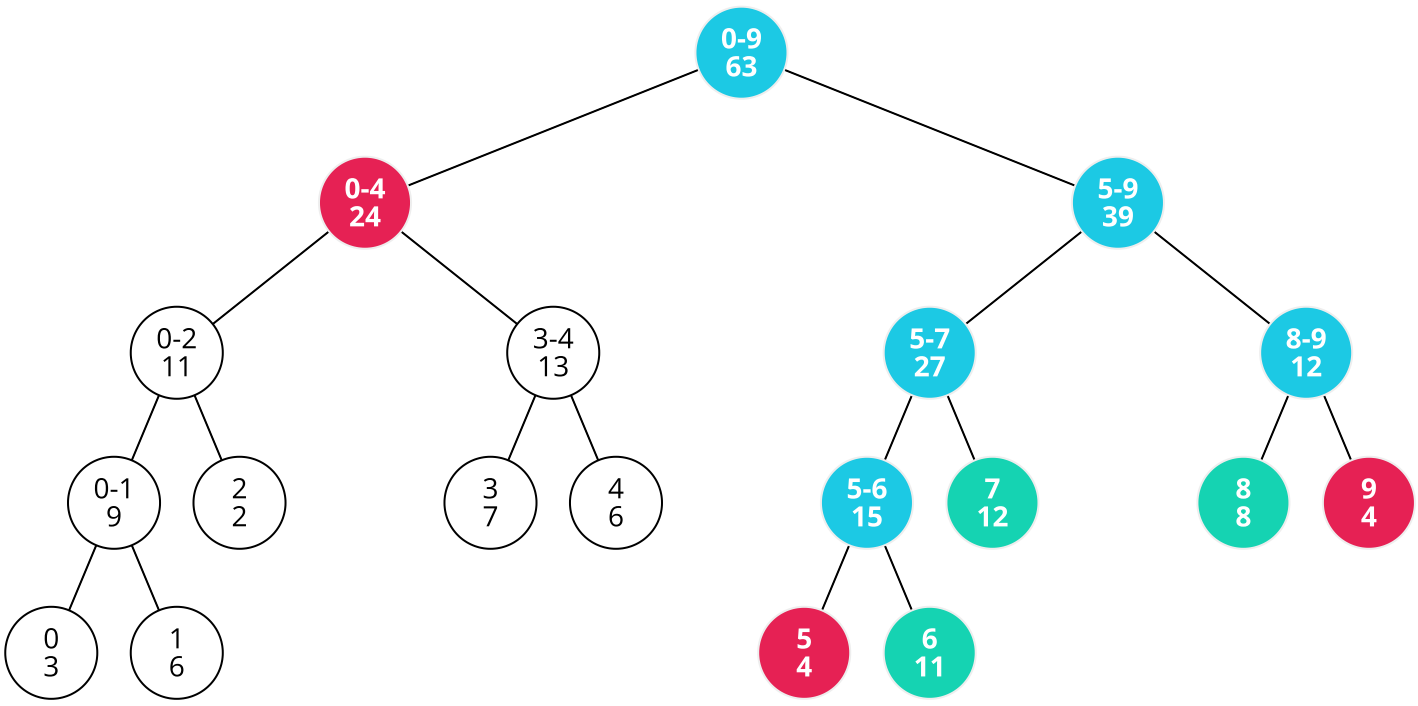
9. 8-9 10 은 `lazy`을 이용해 합을 변경해 8-9 12 가 되고, 두 자식 노드 8 7 와 9 3 에게 `lazy`를 넘겨준 다음 호출합니다.



10. 8 7 에도 `lazy`값이 있으니 합을 변경해 8 8 가 됩니다. `[6,8]`에 포함되니 8을 리턴합니다.

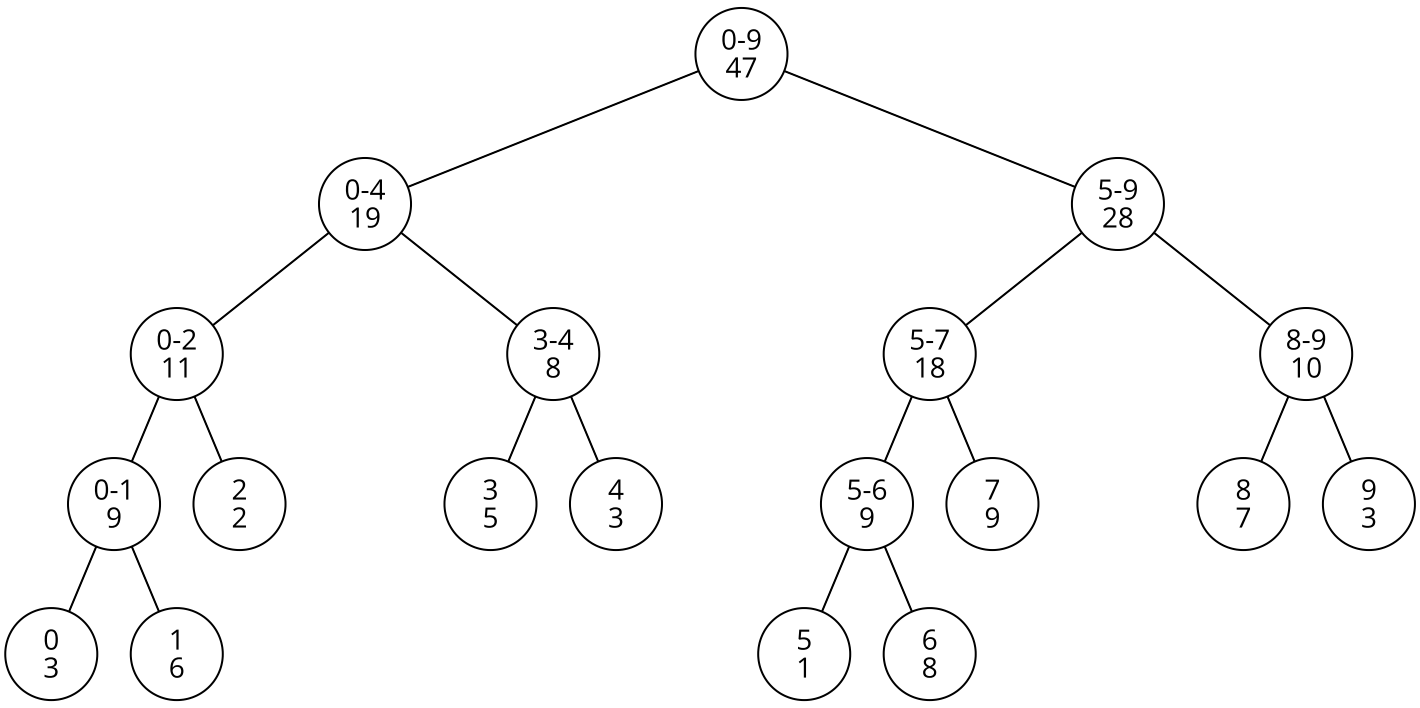
11. 9 3 에는 `lazy`값이 있고, 먼저 합을 변경합니다. 합을 구하려는 구간 `[6,8]`와 겹치지 않으니 0을 리턴합니다.

합을 모두 구한 이후 트리는 다음과 같습니다.



직접 해보기

- $A =$ 3, 6, 2, 5, 3, 1, 8, 9, 7, 3 로 초기화
  - $1 \le N \le 16$
  - $1 \le A[i] \le 99$



i	0	1	2	3	4	5	6	7	8	9
$A[i]$	3	6	2	5	3	1	8	9	7	3

- 변경: `left` = 3, `right` = 7, 더할 값 = 2 실행
- 합 구하기: `left` = 6, `right` = 8 실행

시간 복잡도

수를 변경할 때 방문하는 노드는 합을 구할때 방문하는 노드와 같습니다. 따라서, 두 연산 모두 $O(\lg N)$ 입니다.

10999 - 구간 합 구하기 2

이 글의 가장 처음에 설명한 문제와 같은 문제입니다. 소스 2로 해결할 수 있습니다.

소스 2

C++ Java Python

```python
import sys
import math
input = sys.stdin.readline
def init(a, tree, node, start, end):
    if start == end:
        tree[node] = a[start]
    else:
        init(a, tree, node*2, start, (start+end)//2)
        init(a, tree, node*2+1, (start+end)//2+1, end)
        tree[node] = tree[node*2] + tree[node*2+1]

def update_lazy(tree, lazy, node, start, end):
    if lazy[node] != 0:
        tree[node] += (end-start+1)*lazy[node]
        if start != end:
            lazy[node*2] += lazy[node]
            lazy[node*2+1] += lazy[node]
        lazy[node] = 0

def update_range(tree, lazy, node, start, end, left, right, diff):
    update_lazy(tree, lazy, node, start, end)
    if left > end or right < start:
        return
    if left <= start and end <= right:
        tree[node] += (end-start+1)*diff
        if start != end:
            lazy[node*2] += diff
            lazy[node*2+1] += diff
        return
    update_range(tree, lazy, node*2, start, (start+end)//2, left, right, diff)
    update_range(tree, lazy, node*2+1, (start+end)//2+1, end, left, right, diff)
    tree[node] = tree[node*2] + tree[node*2+1]

def query(tree, lazy, node, start, end, left, right):
    update_lazy(tree, lazy, node, start, end)
    if left > end or right < start:
        return 0
    if left <= start and end <= right:
        return tree[node]
    lsum = query(tree, lazy, node*2, start, (start+end)//2, left, right)
    rsum = query(tree, lazy, node*2+1, (start+end)//2+1, end, left, right)
    return lsum + rsum

n, m, k = map(int, input().split())
a = [int(input()) for _ in range(n)]
h = math.ceil(math.log2(n))
tree_size = 1 << (h+1)
tree = [0] * tree_size
lazy = [0] * tree_size
```

```
m += k
init(a, tree, 1, 0, n-1)
for _ in range(m):
    what, *q = map(int, input().split())
    if what == 1:
        left, right, diff = q
        update_range(tree, lazy, 1, 0, n-1, left-1, right-1, diff)
    else:
        left, right = q
        print(query(tree, lazy, 1, 0, n-1, left-1, right-1))

```

연습 문제

- 16975 - 수열과 쿼리 21
- 1395 - 스위치
- 12844 - XOR